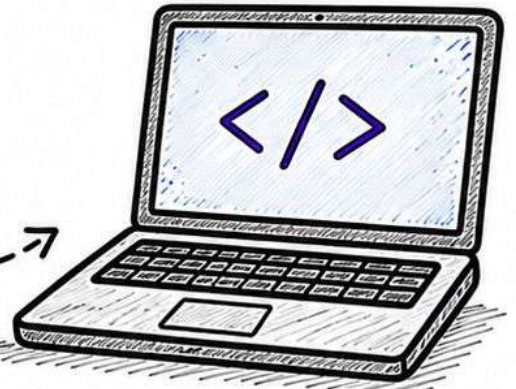
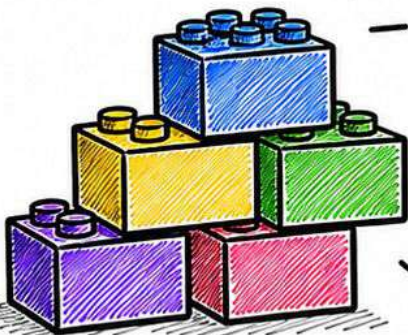


☞ Good code works.
Great code survives change.

⇒ Let's learn ⇐

SOLID PRINCIPLES

in simple words

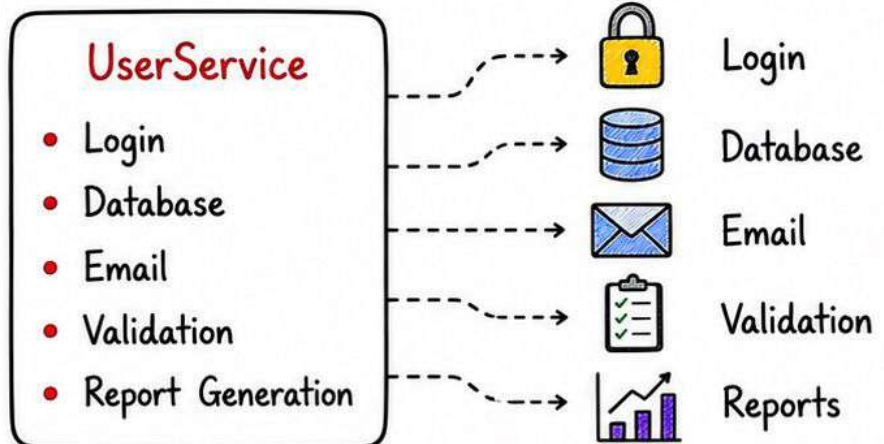


Single Responsibility Principle

"A class should have only ONE reason to change."

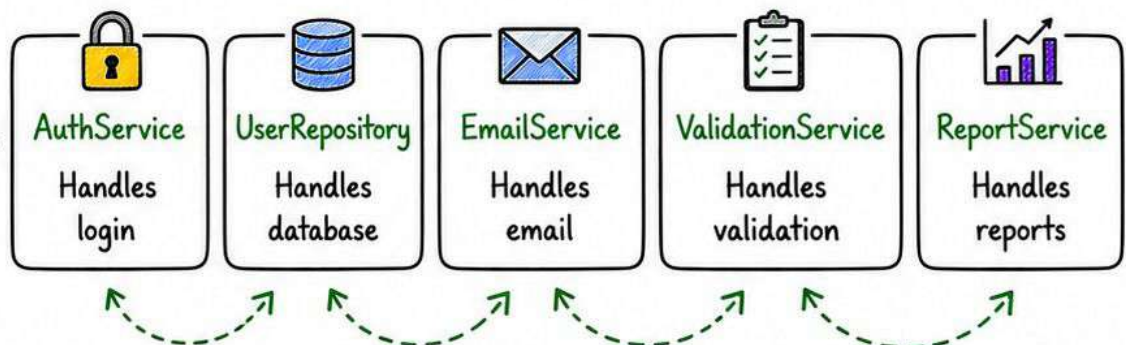
Bad ✗

One class handles multiple things.

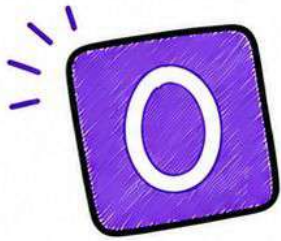


Good ✓

Separate responsibilities into separate classes.



Cleaner code = Easier debugging



Open/Closed Principle



"Open for extension,
closed for modification."



Meaning:

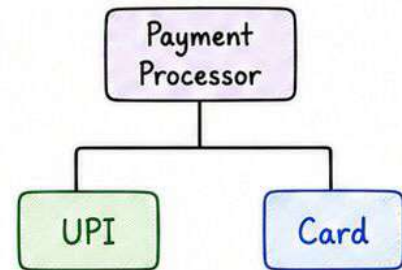
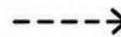
Add new features
WITHOUT changing
old code.

Bad ✗

Every time we add
a new type,
we **modify** existing
code.

PaymentProcessor.java

```
if (type == "UPI")
    processUPI();
else if (type == "Card")
    processCard();
// Adding new type means
// changing this file again!
```



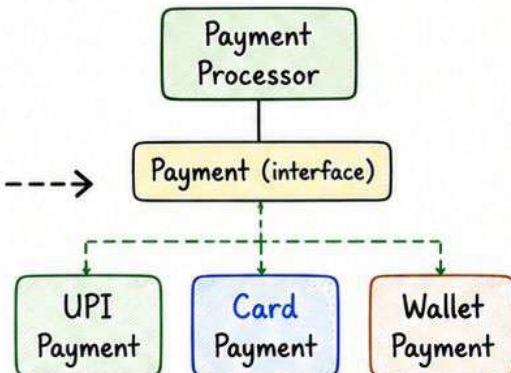
Modify
existing
code

Good ✓

We extend the
system by adding
new classes.
No changes to
existing code.

PaymentProcessor.java

```
public void pay(Payment p) {
    p.pay();
}
```



Extend
without
modifying



Less risk of breaking things





Liskov Substitution Principle



“Child classes should be substitutable for their parent classes.”

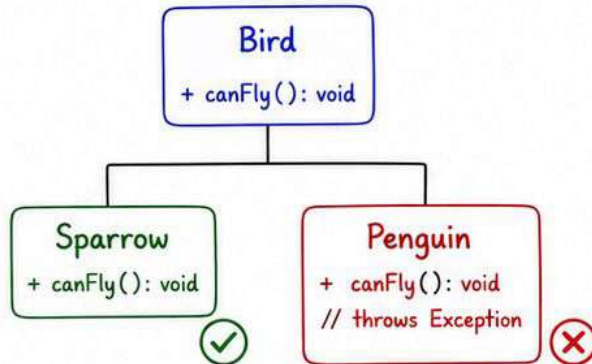


Meaning:

If S is a subtype of T, objects of type S should be replaceable with objects of type T without breaking the behavior.

Bad ✗

Penguin is a Bird, but it **breaks** the expected behavior (**canFly**).



Client Code

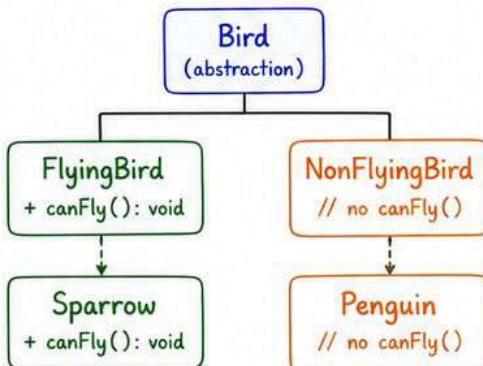
```

void makeBirdFly(Bird bird) {
    bird.canFly();
}

makeBirdFly(new Sparrow()); // ✓ Works
makeBirdFly(new Penguin()); // ✗ Fails
  
```

Good ✓

We design proper hierarchy so that behavior remains consistent.



Client Code

```

void makeBirdFly(FlyingBird bird) {
    bird.canFly();
}

makeBirdFly(new Sparrow()); // ✓ Works
// makeBirdFly(new Penguin()); // ✗ Compile-time error
  
```



Inheritance should make sense, not just look good.



Interface Segregation Principle

“Don't force classes to implement methods they don't use.”

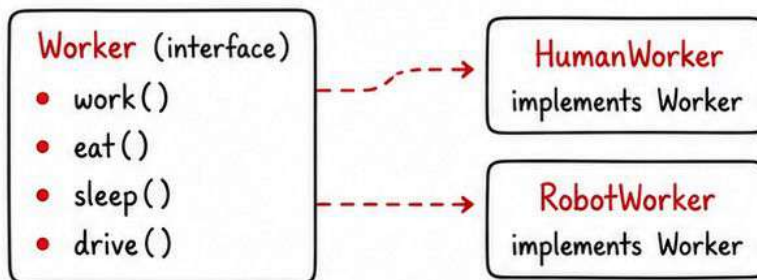


Meaning:

Make interfaces small and specific so classes only implement what they actually need.

Bad X

One fat interface forces classes to implement useless methods.

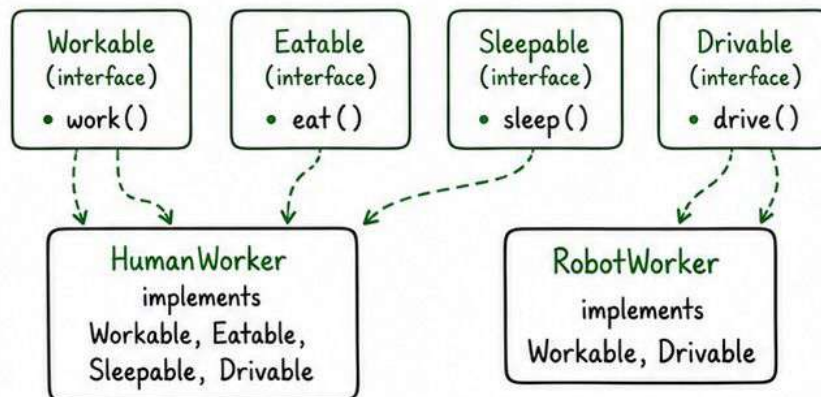


Robot doesn't eat or sleep, but is forced to implement these methods.



Good ✓

Many small interfaces so classes only implement what they need.



Each class implements only the methods it actually needs.



Keep interfaces small & focused.
Better design, easier to maintain.



Dependency Inversion Principle

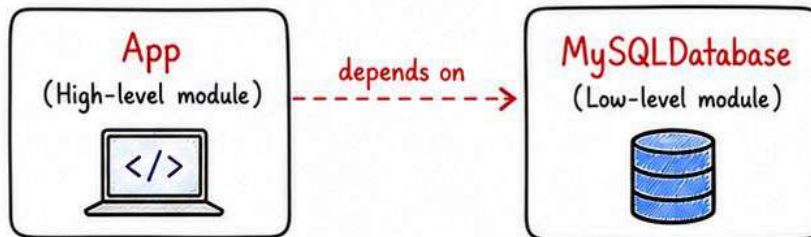
“Depend on abstractions,
not on concrete classes.”

Meaning:

High-level modules should not depend on low-level modules. Both should depend on abstractions.

Bad ✗

High-level module (App) depends on low-level module (MySQL).

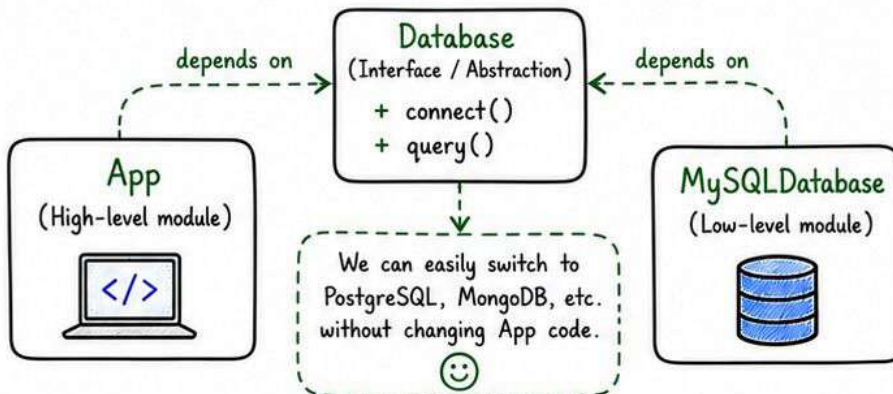


If we want to use another database, we must **change** the App code.



Good ✓

Both high-level and low-level modules depend on abstraction.



Flexible, easy to extend and maintain.



Program to an interface,
not to an implementation.



SOLID

SOLID Principles

7/8

5 principles for writing code that is
maintainable, scalable and easy to change.



S

Single Responsibility Principle

A class should have only **ONE** reason to change.



One class,
one responsibility.
Keep it **focused**.

O

Open/Closed Principle

Open for **extension**,
closed for modification.



Add new features
without changing
existing code.

L

Liskov Substitution Principle

Child classes should be **substitutable**
for their parent classes.



If it looks like the parent
and behaves like the parent,
it should **replace** the parent.

I

Interface Segregation Principle

Don't force classes to implement
methods they don't use.

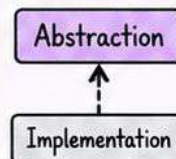


Many **small** interfaces
are better than one
big fat interface.

D

Dependency Inversion Principle

Depend on **abstractions**,
not on concrete classes.



High-level modules and
low-level modules should
depend on **abstractions**.



Follow SOLID, write **clean code**
and build **solid systems**!

